A. Concurrency Control (8 points)

Given the resource hierarchy X(Y(A,B), Z(S,T)), describe the behavior of the following arrival sequence of requests managed by a scheduler that applies hierarchical locking. Suppose that locks are released after the commit of each transaction, which occurs immediately after the last operation of each transaction.

r1(S) w1(A) w2(Z) r2(A) r3(X) w1(Y)

Solution.

Op.	X	Y	A	B	Z	S	T
r1(S)	ISL_1	-	-	-	ISL_1	SL_1	-
w1(A)	IXL_1 (upgr)	IXL_1	XL_1	-	ISL_1	SL_1	-
w2(Z)	$IXL_{1,2}$	IXL_1	XL_1	-	ISL_1 (Conflict - T2 waits!)	SL_1	-
r2(A)	T2 is waiting						
r3(X)	$IXL_{1,2}$ Conflict - T3 waits!	IXL_1	XL_1	-	ISL_1 (Conflict - T2 waits!)	SL_1	-
w1(Y)	$IXL_{1,2}$	XL_1 (upgr)	XL_1	-	ISL_1	SL_1	-
T1 commits	IXL_2	-	-	-	-	-	-
w2(Z)	IXL_2	-	-	-	XL_2	-	-
r2(A)	IXL_2	ISL_2	SL_2	-	XL_2	-	-
T2 commits	-	-	-	-	-	-	-
r3(X)	SL_3	-	-	-	-	-	-
T3 commits	-	-	-	-	-	-	-

B. Physical Databases (8 points)

A table `FILE(fid, name, mediaType, description)` stores 100K tuples on 1K blocks in a primary entry sequenced storage. A table `HYPERLINK(source, target, rel)` stores 1M tuples on 10K blocks in a primary hash built on the primary key with negligible overflow chains. The relation is not symmetric: if A→B is in `HYPERLINK` B→A may or may not be in `HYPERLINK`. We also know that

- val(`mediaType`)=200;
- only 5% of the hyperlinks correspond to a relation of type "alternate".

Consider the query that finds pairs of HTML files, one of which is an alternative version of the other:

```
select F1.fid, F2.fid
from (FILE F1 join HYPERLINK on F1.fid=source) join FILE F2 on F2.fid = target
where F1.mediaType="HTML" and F2.mediaType="HTML" and rel="alternate" and source <> target
```

Describe briefly (but precisely) a reasonable query plan and estimate its cost in the following scenarios:

(a) there are no secondary indexes;
(b) there is also a B+(`mediaType`) index for `FILE` (F=25, 3 levels, 500 leaf nodes).

Solution.

(a) In case there are no secondary indexes, there are two strategies: (1) without caching, and (2) with caching:

(1) Self-join nested loop on `FILE` table + 500×499 hash lookups on `HYPERLINK` (there are 500 HTML files because there are 100k tuples and 200 files per media type, and once a file is chosen, there are only 499 distinct files to choose from). The total cost is:

$$\approx 1\text{k} \times 1\text{k} + (1 \times 250\text{k}) \approx 1.25\text{M}$$

(2) Indexed filtered nested loop with `FILE` table as external and caching. The strategy is the following:

- Find the 500 HTML files (caching their `fid`)
- Table scan (cost: 1k)
- For each pair of HTML files `fid`:
  - Build the hash key from the pair (500×499 key values $\approx$ 250k)
  - Access the primary hash on `HYPERLINK` to check if there is a link (1 block per access $\implies$ 250k)
  - Get the tuple from primary storage and check the `rel` value

The output is formed by the linked pairs with `rel = 'alternate'`, and the total cost is:

$$\approx 1\text{k} + 1 \times 250\text{k} \approx 251\text{k}$$

The plan can actually be improved, because, when caching is available, instead of computing all source-target pairs and use the hash to check the existence of an alternate hyperlink, we can simply scan the 10K blocks of the hash to look for a match with the cached files, so the cost becomes

$$\approx 1\text{k (scan } \texttt{FILE}) + 10\text{k (scan } \texttt{HYPERLINK}) = 11\text{k}$$

(b) Indexed filtered nested loop with `FILE` table as external, optimized with filtering through `mediaType` B+ on `FILE`. The strategy is the following:

- Find the 500 HTML files
  - Lookup the index with search key = `HTML`
  - 2 intermediate nodes + 3 leaf nodes (100K tuples / 500 leaf nodes = 200 pointers per node, so 3 nodes for 500 HTML files) + 500 data blocks for HTML files = 505
  - 500 tuples → get (cache) the `fid` value
- For each pair of HTML files
  - Build the hash key from the pair

- $500 \times 499 \approx 250$k
- Access the primary hash on `HYPERLINK` to check if there is a link
- 1 block per access $\rightarrow$ 250K
- Get the tuple from primary storage and check the relation type `rel`

The output is formed by the linked pairs with `rel = 'alternate'`, and the total cost is:

$$\approx 2 + 3 + 500 + 250\text{k} \approx 250,505$$

As in the previous case, the plan can be improved by simply scanning the 10K blocks of the hash instead of checking all the source-target pairs, with cost

$$\approx 2 + 3 + 500 \text{ (find files on B+)} + 10\text{k (scan } \texttt{HYPERLINK}) \approx 10.5\text{k}$$

C. Triggers (9 points)

The following relational schema offers a simplified description of a university (primary keys underlined):

`Person(Pid, Name)` `Course(Cid, Dept, Credits)`
`Schedule(Teacher, Cid)` `Registration(Student, Cid)`

with obvious foreign keys from `Cid` to `Course(Cid)` and from `Teacher` and `Student` to `Person(Pid)`. The university policy requires the following integrity constraints to hold:

- No one is both a student and a teacher for the same course.
- Every course in the CS department is worth at most 10 credits.
- At least one course in the CS department has at least 5 students.

Design a trigger system that keeps the integrity constraints satisfied by rejecting illegal operations. In particular, the triggers must react to the following kinds of events occurring on the database tables: (*i*) insertions (any table); (*ii*) deletions (any table); (*iii*) updates of `Course`.

Solution.

Triggers for insertions:

```
CREATE TRIGGER insert_schedule
BEFORE INSERT ON Schedule
FOR EACH ROW
BEGIN
    IF EXISTS( SELECT * FROM Registration
               WHERE new.Teacher = Student AND
               new.Cid = Cid )
    THEN
        RAISE EXCEPTION 'Existing student with the same name';
    END IF;
END;

CREATE TRIGGER insert_registration
BEFORE INSERT ON Registration
FOR EACH ROW
BEGIN
    IF EXISTS( SELECT * FROM Schedule
               WHERE new.Student = Teacher AND
               new.Cid = Cid )
    THEN
        RAISE EXCEPTION 'Existing teacher with the same name';
    END IF;
END;

CREATE TRIGGER insert_course
BEFORE INSERT ON Course
FOR EACH ROW
BEGIN
    IF new.Credits > 10 AND new.Dept = 'CS' THEN
        RAISE EXCEPTION 'Too many credits for a CS course';
    END IF;
END;
```

Triggers for deletions:

```
CREATE TRIGGER delete_registration
BEFORE DELETE ON Registration
FOR EACH ROW
BEGIN
```

```
    IF (SELECT dept FROM Course WHERE Cid=old.Cid) = 'CS'
    AND (SELECT COUNT(*) FROM Registration WHERE Cid=old.Cid) = 5
    AND NOT EXISTS( SELECT 1 FROM Registration R JOIN Course C ON R.Cid=C.Cid
                    WHERE dept='CS' AND R.Cid<>old.Cid
                    GROUP BY R.Cid HAVING COUNT(*)>=5 )
    THEN
        RAISE EXCEPTION 'No course left with at least 5 students';
    END IF;
END;
```

Triggers for updates of `Course`:

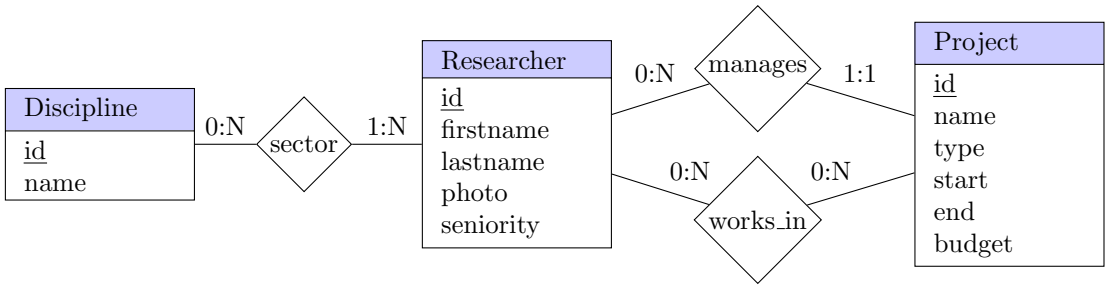
```
CREATE TRIGGER update_course_1
BEFORE UPDATE ON Course
FOR EACH ROW
WHEN NEW.dept = 'CS' -- redundant, already checked in code
BEGIN
    -- same code as trigger insert_course
    IF new.Credits > 10 AND new.Dept = 'CS' THEN
        RAISE EXCEPTION 'Too many credits for a CS course';
    END IF;
END;

CREATE TRIGGER update_course_2
BEFORE UPDATE ON Course
FOR EACH ROW
WHEN new.Dept <> 'CS' AND old.Dept = 'CS'
BEGIN
    -- almost same code as trigger delete_registration
    IF (SELECT COUNT(*) FROM Registration WHERE Cid=old.Cid) >= 5
    AND NOT EXISTS( SELECT 1 FROM Registration R JOIN Course C ON R.Cid=C.Cid
                    WHERE Dept='CS' AND R.Cid<>old.Cid
                    GROUP BY R.Cid HAVING COUNT(*)>=5 )
    THEN
        RAISE EXCEPTION 'No course left with at least 5 students';
    END IF;
END;
```

D. JPA (7 points)

An application lets a researcher edit the projects s/he is responsible for. A project has a name, a start and end date, a type and a budget (an integer number). A researcher can create projects and assign other researchers to them. Researchers have a name, an id, a photo, and a seniority level (a string). Researchers also pertain to one or more disciplines (e.g., computer science, automation, telecommunications, etc.). The application lets a researcher access the list of projects s/he manages (in descending order of end date) and the list of projects s/he works in. By selecting one project s/he can see the details of the project including the list of the researchers that work in it/manage it with the disciplines they belong to. Researchers are in the order of hundreds, projects are in the order of thousands and disciplines in the order of tens.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.



Solution.

See separate pdf file.